

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Настройка системы SOPS

Выполнил обучающийся:

Груздев Николай Александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Получить практический опыт по настройке SOPS.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Выполним установку инструмента SOPS (рис. 1).

```
root@vm2:~# curl -LO https://github.com/getsops/sops/releases/download/v3.11.0/sops-v3.11.0-linux.amd64
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 44.1M 100 44.1M    0     0 20.5M      0  0:00:02  0:00:02 --:--:-- 31.1M
```

Рисунок 1 – Установка SOPS

Сгенерируем ключ шифрования с помощью GPG, указав имя, email и пароль (рис. 2).

```
user@vm2:~$ gpg --gen-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/user/.gnupg'
gpg: создан щит с ключами '/home/user/.gnupg/pubring.kbx'
Замечание: "gpg --full-generate-key" вызывает полнофункциональный диалог создания ключа.

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Debian
Адрес электронной почты: some@mail.com
Вы выбрали следующий идентификатор пользователя:
"Debian <some@mail.com>"
```

Рисунок 2 – Генерация GPG-ключа

Экспортируем отпечаток ключа в переменную окружения SOPS_PGP_FP (рис. 3).

```
user@vm2:~/sops$ export SOPS_PGP_FP="6488A8F165E4F898033AE5F77ABCE6077CCF8420"
```

Рисунок 3 – Настройка переменной окружения

Зашифруем YAML-файл пайплайна из предыдущей работы (рис. 4).

```
user@vm2:~/sops$ sops encrypt gitlab.yaml > enc.gitlab.yaml
```

Рисунок 4 – Шифрование YAML-файла

После шифрования все значения в файле заменены на зашифрованные данные (рис. 5).

```

stages:
- ENC[AES256_GCM,data:NdG8Jw==,iv:GB9cwz4WgFP+DAoRKUM0dahJ87dP0pNwY0yCbfnIQ>
- ENC[AES256_GCM,data:etyh1fg=,iv:hDDj61kbXds7sZuLAgHaX4w1ctTYsLVXWPmoT//+8>
- ENC[AES256_GCM,data:krZ+w7uU,iv:y+0BNbQ/Bt5XJlQd6kvXwCbF0XzNETeIP/q1SxzXV>
pytest:
stage: ENC[AES256_GCM,data:qR0fQQ==,iv:FF9ATjQ7Lk/QCkIWzpmA/zDsQECIVGlj94cJ>
before_script:
- ENC[AES256_GCM,data:40eBrJBvDRUv8Mz+R6TE0lKHgyq2rCPyrsiyjVbu/3bISZdZk>
- ENC[AES256_GCM,data:I50/g60WgxK5eRC3J/tnvX4Q1ADU9BomYa+K3QUAI75Lkyt0p>
script:
- ENC[AES256_GCM,data:hoolNvaemC8tpD/8LjcnivShl0XoBhlZY6+n/DPoemlyH9yN8>
- ENC[AES256_GCM,data:fALdwiBPgXq+Fau0Uftokdk5eYJ7ijTdwYyhti+uUdEPjN5r>
- ENC[AES256_GCM,data:13iAH9jSQ6jenk4rNwSbfUua/c9rNjc8JKzhE3KmFo12AJGNk>
artifacts:
when: ENC[AES256_GCM,data:88sC/U42,iv:+NuwnwW5X4JJ4EEPnrTh+lmNnx1R5uw00>
reports:
junit: ENC[AES256_GCM,data:ENPdbqMcBKbeIg==,iv:1+T0auzghSe30qUBoCmP>
compile_crystal:
stage: ENC[AES256_GCM,data:jFHIGQM=,iv:36B/JA3rIpG/IWxgFmqXBr3DVLSPra8AkK1T>
before_script:

```

Рисунок 5 – Зашифрованный YAML-файл

Выполним расшифровку файла для проверки содержимого (рис. 6).

```

user@vm2: ~/sops$ sops decrypt enc.gitlab.yaml
stages:
- test
- build
- deploy
pytest:
stage: test
before_script:
- echo "Установка необходимых библиотек..."
- pip install pytest pytest-cov -r requirements.txt
script:
- echo "Выполнение тестирования с помощью pytest с выводом отчёта..."
- python -m pytest --cov --junitxml=report.xml
- echo "Тестирование завершено."
artifacts:
when: always
reports:
junit: report.xml
compile_crystal:
stage: build
before_script:
- echo "Установка Nuitka..."
- pip install Nuitka
script:

```

Рисунок 6 – Расшифрованный файл

Создадим второй GPG-ключ (рис. 7).

```

user@vm2: ~/.gnupg$ gpg --gen-key
gpg (GnuPG) 2.2.40; Copyright (C) 2022 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Замечание: "gpg --full-generate-key" вызывает полнофункциональный диалог создани
я ключа.

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Debian2
Адрес электронной почты: some@mail.com
Вы выбрали следующий идентификатор пользователя:
"Debian2 <some@mail.com>"

```

Рисунок 7 – Создание второго ключа

Добавим новый ключ в список ключей зашифрованного файла (рис. 8).

```
user@vm2:~/sops$ sops rotate -i --add-pgp E96A92DA2E3693E6D3E3C92078C16B4A547751
20 enc.gitlab.yaml
```

Рисунок 8 – Добавление второго ключа в файл

Теперь файл защищён двумя ключами (рис. 9).

```
pgp:
- created_at: "2025-12-29T16:09:58Z"
  enc: |-
    -----BEGIN PGP MESSAGE-----

    hQGMA6TEh/DFF3DpAqv/TBBAIsxgNeH1IkZJmauVNjoxHfEtjQkEX38mRWUdIyEJ
    iI3NS1A2bd0/WlJjqcaK4zgp95X6DKugUQTgsbYwwi5jRQ8hGf4EHII/YuWyqXdg
    MNAMroyVDanPmtEPpcOJG156TiAjqBwekUPo83tZtJKxrI5Ew/kHPxE9XQw8jg
    OeF6pFYyEwYZWbOswRhh08i/01+FeWyFf00H5S8oApQ7L4LwapJSzVcocGn/hzKV
    9DKw9Ix75/fqbaK096mQ2606iZyf0ne1cSpdeISyZ0zehVCqw1Xbm4pkh4tfuYSY
    aJQcEPJXnMHDIQ7VZugybRi55ey+oKIn13T/w56e1Tp8c9QgWEqIicmhti2jiEu
    v1k0Cgp3gh4wZ/iklk0pL1FP8SX53B5M0v4gs6MsiSvCqtGn7Lv0u9Dx4L5H3z8d
    nWR73Qn5FCEW+ISVyCLVs+A1Ck0jBy0i7GWS0xhkFp9emoLQufgxu40Ixg1P+jQ/
    wuwrjB576oimou7Dp6me0lwBjLj1Sz+gbDYr5iWcNUVHnN5EYfb9hTWlVNhBX5Wv
    W/15RQbgw0zQEH+e0bTwc2p62pSmj077MYZjwu2nCLo45QeqWctF4+Pk6Y48QnWT
    5Lt+Y50NrN2wusvieg==
    =02t2
    -----END PGP MESSAGE-----
  fp: 6488A8F165E4F898033AE5F77ABCE6077CCF8420
- created_at: "2025-12-29T16:23:57Z"
  enc: |-
    -----BEGIN PGP MESSAGE-----
```

Рисунок 9 – Список ключей файла

Удалим из файла первый ключ (рис. 10).

```
user@vm2:~/sops$ sops rotate -i --rm-pgp 6488A8F165E4F898033AE5F77ABCE6077CCF8420
0 enc.gitlab.yaml
```

Рисунок 10 – Удаление первого ключа

Теперь файл использует только второй ключ (рис. 11).

```
sops:
  lastmodified: "2025-12-29T16:27:01Z"
  mac: ENC[AES256_GCM,data:l1InqEnkD5TEBLPn8jmV2d+iV3KRbFaHJcuCA78Cg4p+VZFko5
  pgp:
    - created_at: "2025-12-29T16:26:56Z"
      enc: |-
        -----BEGIN PGP MESSAGE-----

        hQGMA3xMwK7x1K0VAQwAofUMYv+x8FSB8X6UJWkaqkEKTUhm417UCouV7mbqy3pr
        f2bUsFMFT6aUkRQJwmJpRnmOmKQsSVK+4sPC37vG6F92f9IMjbBgqhc6m+BE6NJ
        mXZx1Io+FEZK/YcW6XJ/dpf959wvo8Cn0fekLRVepPA1e4cbqNCXYDOU+m1xixq5
        5Q5oGBqzHfUVAUg8KHL1VsSSbfsktn1EK3LgasaDEe3q324QLOcrAtRHh+pxx1Nd
        INu+S5v0668yUDf5IXoNvsrNpzA2odwml8uwYF5+3juNyThJFoI1hxHHigT29yxE
        iS1/xa3Xs5dc1A6upoxq9fUTszW0Zzfx4G2s3/d9KZBkwnPtldMqnewuTecKftmS
        Hr7yZ3azzdzjnboT/AHbeyWG8/rVeVMempv50r+Ja+HJUA37wQlqHVEpoaqOyK7C
        sRkWLYKtfrXBT/5HegwPGRe0HCuutLdauCqbYXOXZYamziOzoeEMA1zDGjULkGcB
        3E1x4TTrMBETala53MR3014BRHPqIXUD0rYXF+8spA9zk19v71z19nnQmgA1t+A
        igM+gUZIifLgEPiSmT9jIKAxK9a1VRdvq19Y3WdyK3FNEDDfdImqWDM0ryZUUI
        NbbWPTOLbOPc3GaHNdEo
        =2xya
        -----END PGP MESSAGE-----
      fp: E96A92DA2E3693E6D3E3C92078C16B4A54775120
  unencrypted_suffix: _unencrypted
  version: 3.11.0
```

Рисунок 11 – Обновлённый список ключей

Выполним частичное шифрование файла, указав для шифрования только секцию pytest (рис. 12).

```
user@vm2:~/sops$ sops encrypt --encrypted-regex '^(pytest)$' gitlab.yaml > part
nc.gitlab.yaml
```

Рисунок 12 – Частичное шифрование YAML-файла

После операции зашифрована только выбранная секция (рис. 13).

```
stages:
  - test
  - build
  - deploy
pytest:
  stage: ENC[AES256_GCM,data:BIzm0Q==,iv:dypz5p4DKL3yU/LScdWIREMQpsVyIOu+wfH
  before_script:
    - ENC[AES256_GCM,data:qr0TwtZBNCNSuTXwUshuTMNMFMBjbu5HOjDoigRRZq73xvG8
    - ENC[AES256_GCM,data:ZLHofd+VUBbaKqzmYwU05+t7r5/vIAUHyEWApoZEFEj10qYX
  script:
    - ENC[AES256_GCM,data:dUeBTHiJMDhiQ0/ADyZE80LRs9RVB1wGnTAn6oYokkT9WhFf
    - ENC[AES256_GCM,data:i+wp/VXoe59UjHKCUqy0gWShOv0GgnRmyzt7x95ZtKOILmYc
    - ENC[AES256_GCM,data:vDBohIxfG8MeaqRBxsncwm8ZrbssRiXNCOCE16B7RD9aQYXh
  artifacts:
    when: ENC[AES256_GCM,data:A/alM/1Q,iv:WyfoPao9JCKsIafj6aFH1P/mZNIJrzB7
    reports:
      junit: ENC[AES256_GCM,data:oSXsaszo2oCKsA==,iv:TLgE1NUBopBQ/HsTMFM
  compile_crystal:
    stage: build
    before_script:
      - echo "Установка Nuitka..."
      - pip install Nuitka
    script:
      - echo "Выполнение сборки..."
```

Рисунок 13 – Зашифрованный раздел pytest

Создадим тестовые зашифрованные файлы в разных форматах: JSON, ENV и INI (рис. 14-16).

```

"hello": "ENC[AES256_GCM,data:qIj5xryghIZC8D9TfBDmJ0uQAtYf/g9hVHmMRJBtH>
"example_key": "ENC[AES256_GCM,data:40BjHDg82ilSSI1wUg==,iv:sBIEALisnPW>
"example_array": [
  "ENC[AES256_GCM,data:5cHf0pQdtqB6IZoAfyQ=,iv:DBpc3Fq0pkunxVHRco>
  "ENC[AES256_GCM,data:ZyHkvZe/TnU02R7EkDY=,iv:xRCd8CF2+OVmh+k0a4>
],
"example_number": "ENC[AES256_GCM,data:fvNlR5vWIXZ7+DIX,iv:0bspy0CNVkvQ>
"example_booleans": [
  "ENC[AES256_GCM,data:oCv7Ew==,iv:x2J+LpQUwrx7jfvkI3GcPmbXjUXovT>
  "ENC[AES256_GCM,data:8U/ByZw=,iv:KT1j0PDQD12tfAWG6v/7lU9xPQzGEM>
],
"sops": {
  "lastmodified": "2025-12-29T17:10:38Z",
  "mac": "ENC[AES256_GCM,data:PmsU+FXf2Fy0C3FEK24ot0DI2/xfStc+Xrn>
  "pgp": {
    {
      "created_at": "2025-12-29T17:10:10Z",
      "enc": "-----BEGIN PGP MESSAGE-----\n\nhQGMA6TE>
      "fp": "6488A8F165E4F898033AE5F77ABCE6077CCF8420"
    }
  }
}

```

Рисунок 14 – Зашифрованный test.json

```

#ENC[AES256_GCM,data:0+uZZ/MDK9H75wHU012RIjYQ9J8WzUUQtg==,iv:0cEwkyYjBt+Rxaay5Z>
hello=ENC[AES256_GCM,data:znB81ZzsJ+5dciDYtvB42qz1T7zF7P6/+nbjZ5k5ffrV/PPju5Y+>
example_key=ENC[AES256_GCM,data:PUH6GkbGea4/WoWlrgg==,iv:H9aKgKi2ZW2p60j1JI/a2de>
example_multiline=ENC[AES256_GCM,data:T+gM/e6RCj1f0tU=,iv:iRIMfi3nXinJQqg2egBle>
sops_lastmodified=2025-12-29T17:11:32Z
sops_mac=ENC[AES256_GCM,data:39b20wbLip27Tu3z6b6pRuHzMcOoDHorMe5zHyGheJqrA6hDMh>
sops_pgp__list_0__map_created_at=2025-12-29T17:11:07Z

```

Рисунок 15 – Зашифрованный test.env

```

; ENC[AES256_GCM,data:3PJrIzxREW0qJWGuTv14QvmoCGth7q7J,iv:mQdgdztBUTsFEQNNTjMyb>
[Welcome!]
hello = ENC[AES256_GCM,data:+UbdZUAJSNSFMnEqztzTNFKm1xcXgBmqmw3e/VFLZyVwfZ>
example_key = ENC[AES256_GCM,data:04d6P8tdStsfy8Zg4w=,iv:3dJomUX5B12MinI8qliXu>

[sops]
pgp__list_0__map_created_at = 2025-12-29T17:48:28Z
pgp__list_0__map_enc = -----BEGIN PGP MESSAGE-----\n\nhQGMA6TEx/DFf3DpAQ>
pgp__list_0__map_fp = 6488A8F165E4F898033AE5F77ABCE6077CCF8420
unencrypted_suffix = _unencrypted
version = 3.11.0

```

Рисунок 16 – Зашифрованный test.ini

Добавим второй GPG-ключ в группу ключей для файла test.ini (рис. 17-18).

```

user@vm2:~/sops$ sops groups add --file test.ini --pgp E96A92DA2E3693E6D3E3C9207
8C16B4A54775120 > test_group.ini

```

Рисунок 17 – Добавление группы ключей


```

ENC[AES256_GCM,data:3PJrIzxrEW0qJWGuTvi4QvmoCGth7q7J,iv:mQdgdztBUTsfEQNNTjMyb>
[Welcome!]
```

```

hello      = ENC[AES256_GCM,data:+UbdZUAJSNSFMnEqztzNFKm1xcXgBmqmw3e/VFLZyVWfZ>
example_key = ENC[AES256_GCM,data:04d6P8tdStsfFy8Zg4w=,iv:3dJomUX5B12MinI8Q1iXu>

[sops]
mac = ENC[AES256_GCM,data:aAurY>
key_groups__list_1__map_pgp__list_0__map_fp = E96A92DA2E3693E6D3E3C9207>
shamir_threshold = 2
key_groups__list_0__map_pgp__list_0__map_enc = -----BEGIN PGP MESSAGE----->
key_groups__list_1__map_pgp__list_0__map_enc = -----BEGIN PGP MESSAGE----->
lastmodified = 2025-12-29T17:48:32Z
unencrypted_suffix = _unencrypted
version = 3.11.0
key_groups__list_1__map_pgp__list_0__map_created_at = 2025-12-29T17:48:53Z
key_groups__list_0__map_pgp__list_0__map_fp = 6488A8F165E4F898033AE5F77>

```

Рисунок 18 – Файл с несколькими ключами

Настроим аудит изменений зашифрованных файлов. Для этого установим PostgreSQL (рис. 19).

```

root@vm2:~# apt-get install postgresql
Чтение списков пакетов... Готово
Построение дерева зависимостей... Готово
Чтение информации о состоянии... Готово
Будут установлены следующие дополнительные пакеты:
  libcommon-sense-perl libjson-perl libjson-xs-perl libllvm14 libpq5
  libtypes-serialiser-perl postgresql-15 postgresql-client-15
  postgresql-client-common postgresql-common sysstat
Предлагаемые пакеты:
  postgresql-doc postgresql-doc-15 isag

```

Рисунок 19 – Установка PostgreSQL

Войдём под пользователем postgres и создадим пользователя user (рис. 20).

```

postgres@vm2:~$ createuser -P --interactive
Введите имя новой роли: user
Введите пароль для новой роли:
Повторите его:
Должна ли новая роль иметь полномочия суперпользователя? (y - да/n - нет) y

```

Рисунок 20 – Создание пользователя БД

Запустим интерактивную консоль PostgreSQL и создадим базу данных (рис. 21).

```

postgres=# CREATE DATABASE db;
CREATE DATABASE

```

Рисунок 21 – Создание базы данных

Импортируем схему `schema.sql` из репозитория SOPS в созданную БД (рис. 22).

```
user@vm2:~/sops$ psql -U user -d db < schema.sql
CREATE TABLE
CREATE ROLE
GRANT
```

Рисунок 22 – Импорт SQL-схемы

Создадим директорию `/etc/sops` и поместим в неё файл `audit.yaml` с параметрами подключения к базе данных (рис. 23).

```
backends:
  postgres:
    - connection_string: "postgres://sops:sops@localhost/db"
```

Рисунок 23 – Конфигурационный файл `audit.yaml`

Несколько раз отредактируем зашифрованные файлы, после чего проверим журнал изменений в таблице `audit_event` (рис. 24).

```
postgres@vm2:~$ psql
psql (15.14 (Debian 15.14-0+deb12u1))
Введите "help", чтобы получить справку.

postgres=# \c db
Вы подключены к базе данных "db" как пользователь "postgres".
db=# SELECT * FROM audit_event;
```

Рисунок 24 – Запрос к таблице `audit_event`

В таблице сохранены записи о всех операциях с файлами (рис. 25).

id	timestamp	action	username	file
1	2025-12-29 22:47:49.20303	decrypt	user	/home/user/sops/test.json
2	2025-12-29 22:47:59.336956	encrypt	user	/home/user/sops/test.json
3	2025-12-29 22:48:06.998096	decrypt	user	/home/user/sops/test.env
4	2025-12-29 22:48:11.945614	encrypt	user	/home/user/sops/test.env

(4 строки)

Рисунок 25 – Журнал изменений зашифрованных файлов

ВЫВОД

В ходе работы были получены практические навыки по настройке и использованию инструмента SOPS для шифрования конфиденциальных данных, включая управление ключами, избирательное шифрование файлов разных форматов и организацию аудита изменений.